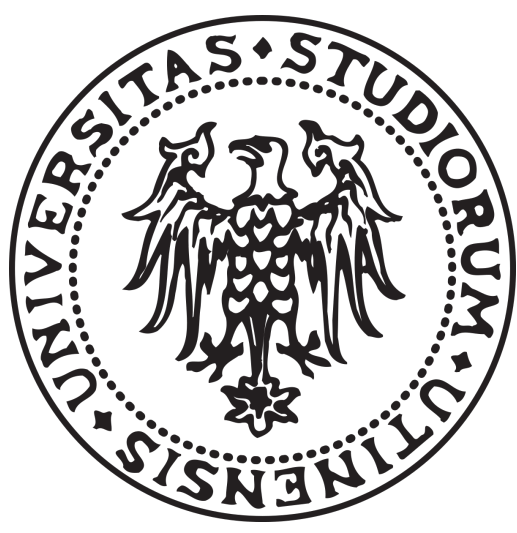




INGEGNERIA DEL SOFTWARE – 2025–26

DIAGRAMMA DI ATTIVITÀ

LEZIONE 19
01/12/2025
VINCENZO RICCIO

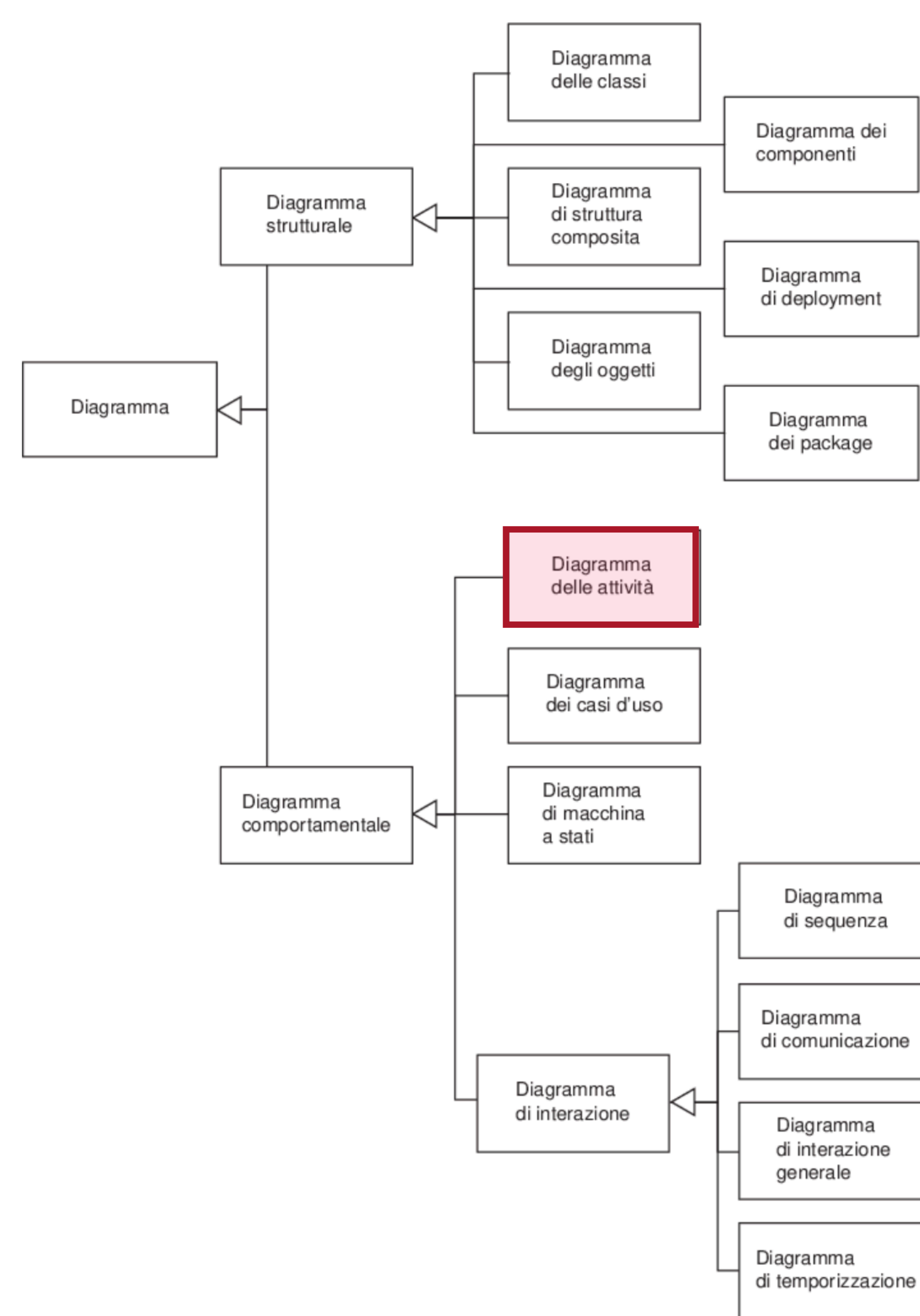


- Fowler - Capitolo 11



DIAGRAMMI DI ATTIVITÀ

- ▶ Gli activity diagram permettono di modellare la logica procedurale di un algoritmo ma anche processi di business e workflow esistenti nella realtà da modellare
- ▶ Tali diagrammi sono un'estensione dei diagrammi di flusso che introduce anche le esecuzioni concorrenti





ACTIVITY DIAGRAM: NOTAZIONE



- Un'attività corrisponde a un insieme coerente di azioni elaborative compiute dal sistema
- Un'azione é un'unità discreta di lavoro atomica all'interno dell'attività
- Un'azione é rappresentata in UML come un rettangolo dai bordi arrotondati, etichettato con il nome/descrizione dell'azione stessa
- Un activity diagram rappresenta una singola attività composta da azioni

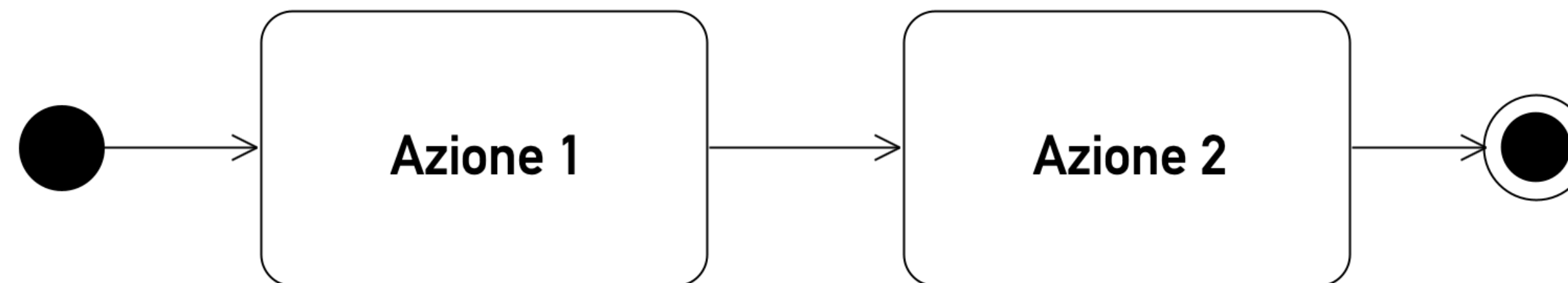


Azione 1

A diagram of a UML action node, which is a rounded rectangle with a black border. The text "Azione 1" is centered inside the rectangle in a bold, black, sans-serif font.

- Il flusso di controllo tra le attività é modellato attraverso frecce entranti/uscanti che collegano le attività
- Per convenzione, l'ingresso nell'azione coincide con il suo inizio, l'uscita dall'azione coincide con il suo completamento
- I nodi iniziali e finali indicano rispettivamente l'inizio e la fine dell'elaborazione
- Se esistono più nodi iniziali in una attività, i flussi cominciano simultaneamente in tutti i nodi iniziali e vengono eseguiti in parallelo

INDICA DOVE INIZIA
IL FLUSSO QUANDO
VIENE INVOCATA
UN'ATTIVITÀ

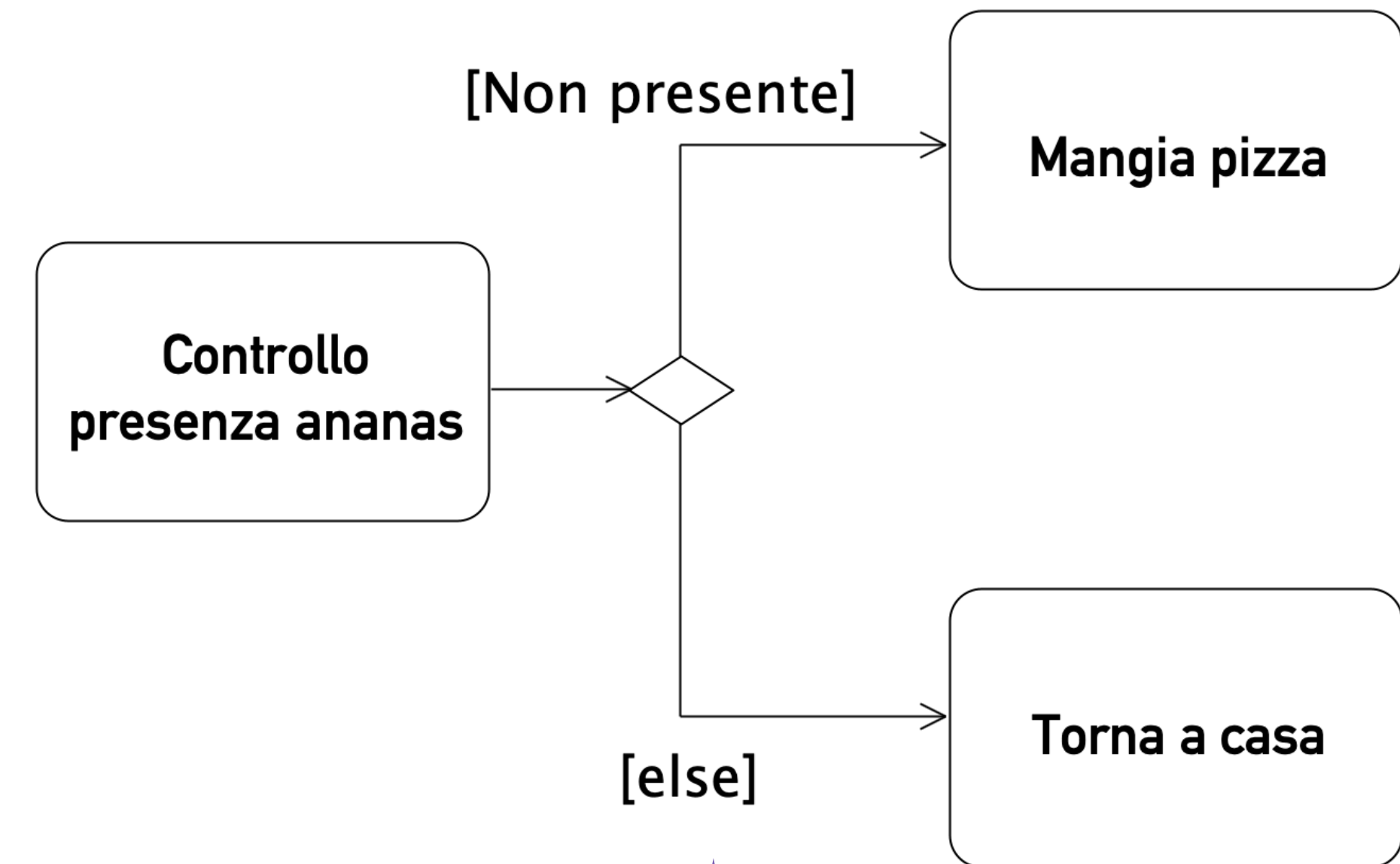


IL PRIMO NODO
FINALE AD ESSERE
ATTIVATO FERMA TUTTI
I FLUSSI ALL'INTERNO
DI UN'ATTIVITÀ

DECISIONE (BRANCH)



- Azione di decisione che corrisponde a un IF
- Rombo con un solo arco entrante e più archi uscenti
- Ciascun arco uscente é etichettato con una condizione booleana di guardia tra parentesi quadre
- Le condizioni di guardia devono essere tra loro mutuamente esclusive
- Dopo una decisione si deve intraprendere uno e uno solo dei flussi uscenti

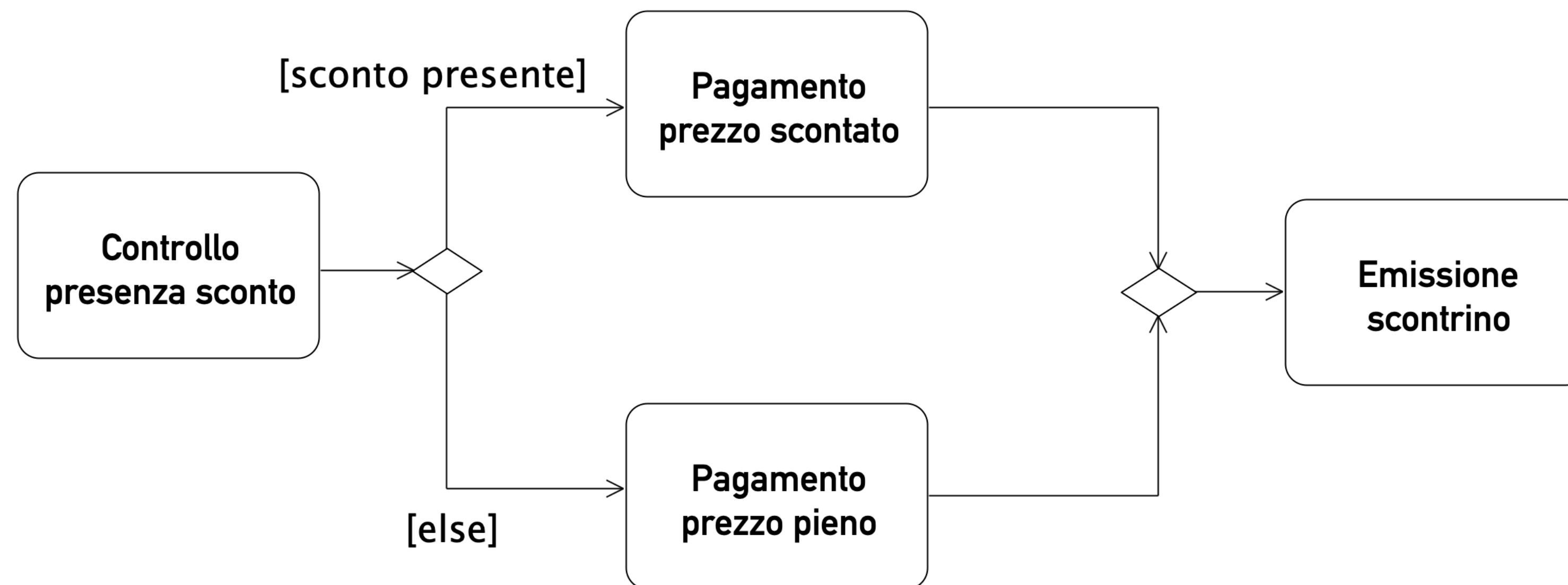


LA CONDIZIONE ELSE INDICA IL CAMMINO DA INTRAPRENDERE QUANDO NESSUNA DELLE CONDIZIONI PRECEDENTI È VERIFICATA. NON ESISTE ELSE IMPLICITO PERCHÉ BISOGNA SEMPRE INDICARE UN POSSIBILE FLUSSO IN USCITA

GIUNZIONE (MERGE)



- Segnala la fine del comportamento condizionale iniziato con una decisione
- Rombo con più flussi di input e un solo flusso di output

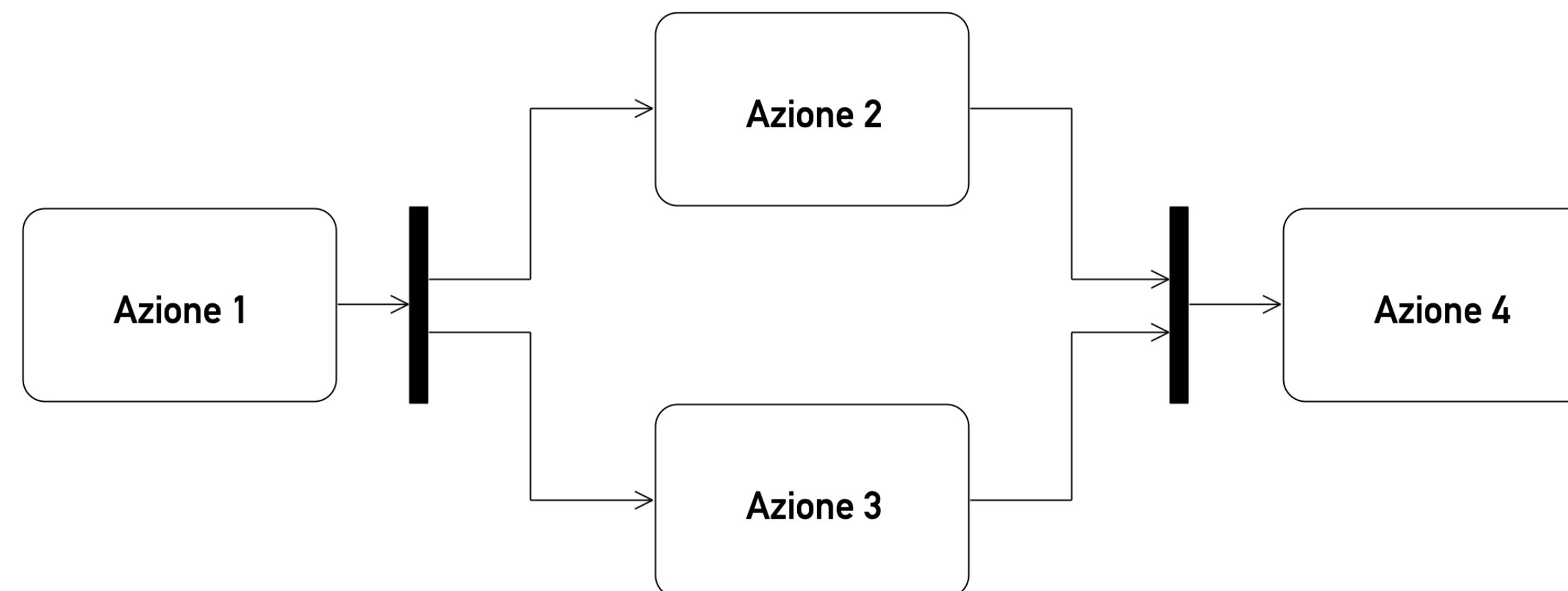


IL MERGE IMPLICITO (RIPORTARE PIÙ NODI ENTRANTI SENZA SIMBOLO DI GIUNZIONE QUANDO SI DESCRIVE UN COMPORTAMENTO CONDIZIONALE) NON FUNZIONA PIÙ IN UML2. DA EVITARE PER NON CREARE CONFUSIONE

FORK E JOIN

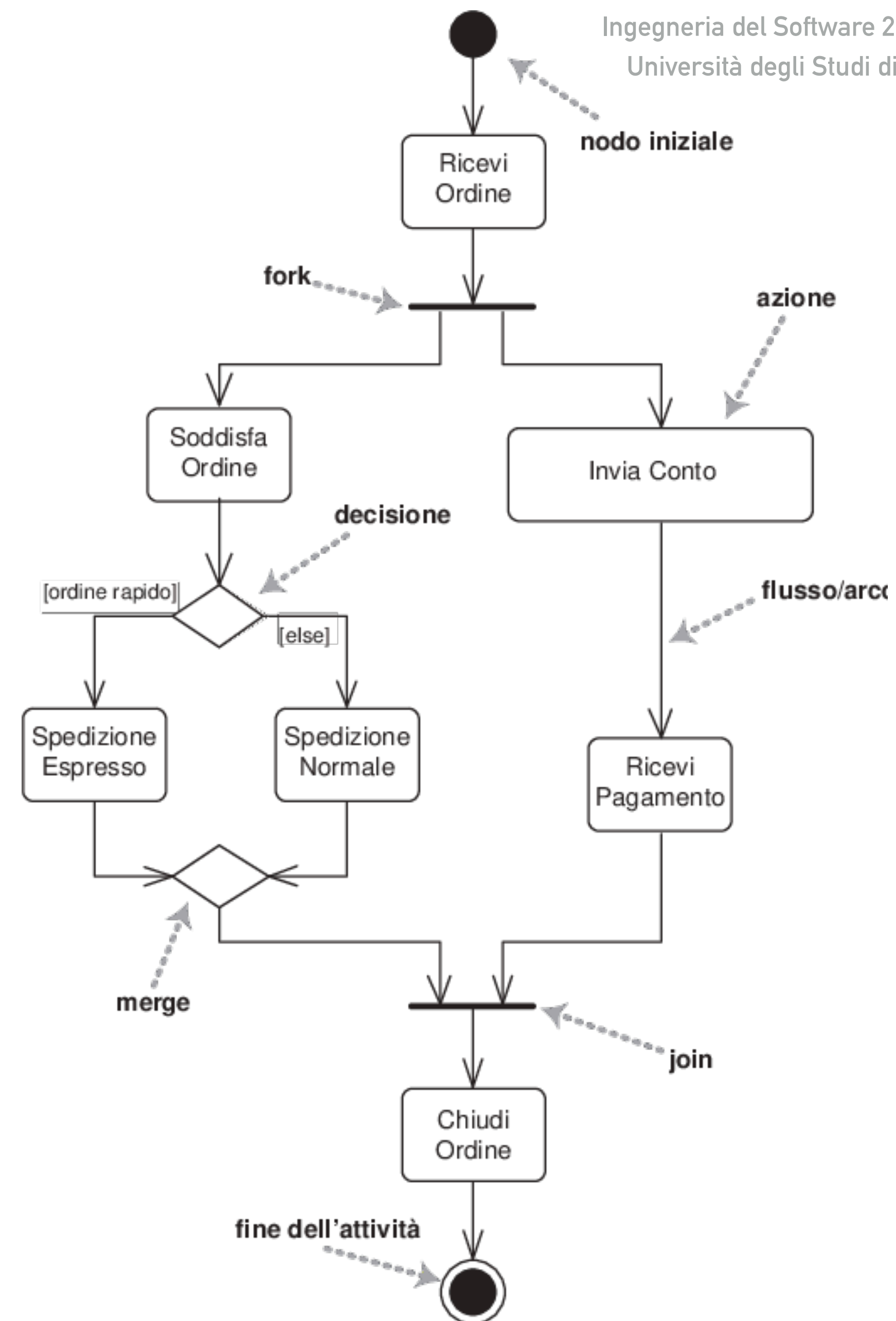


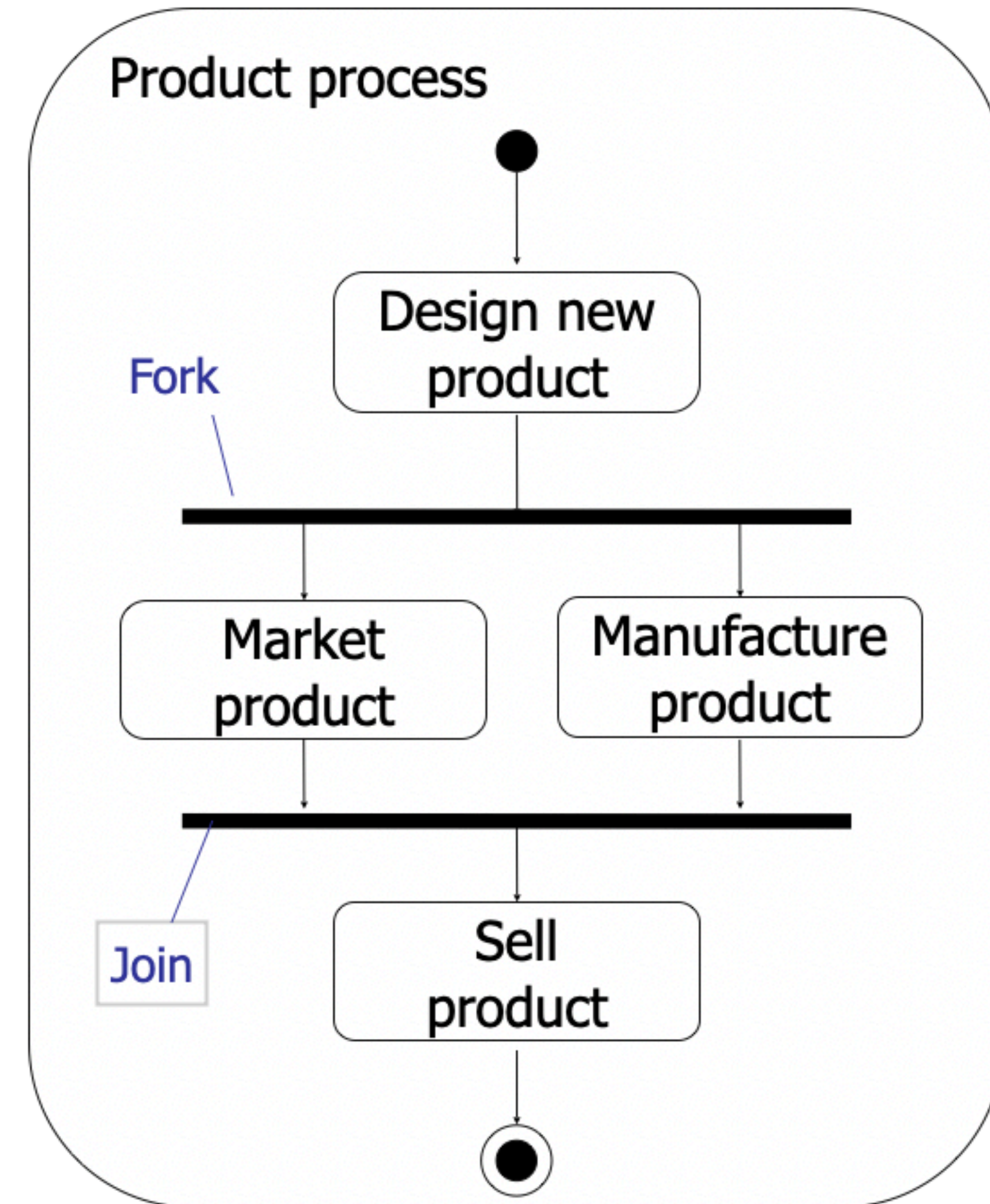
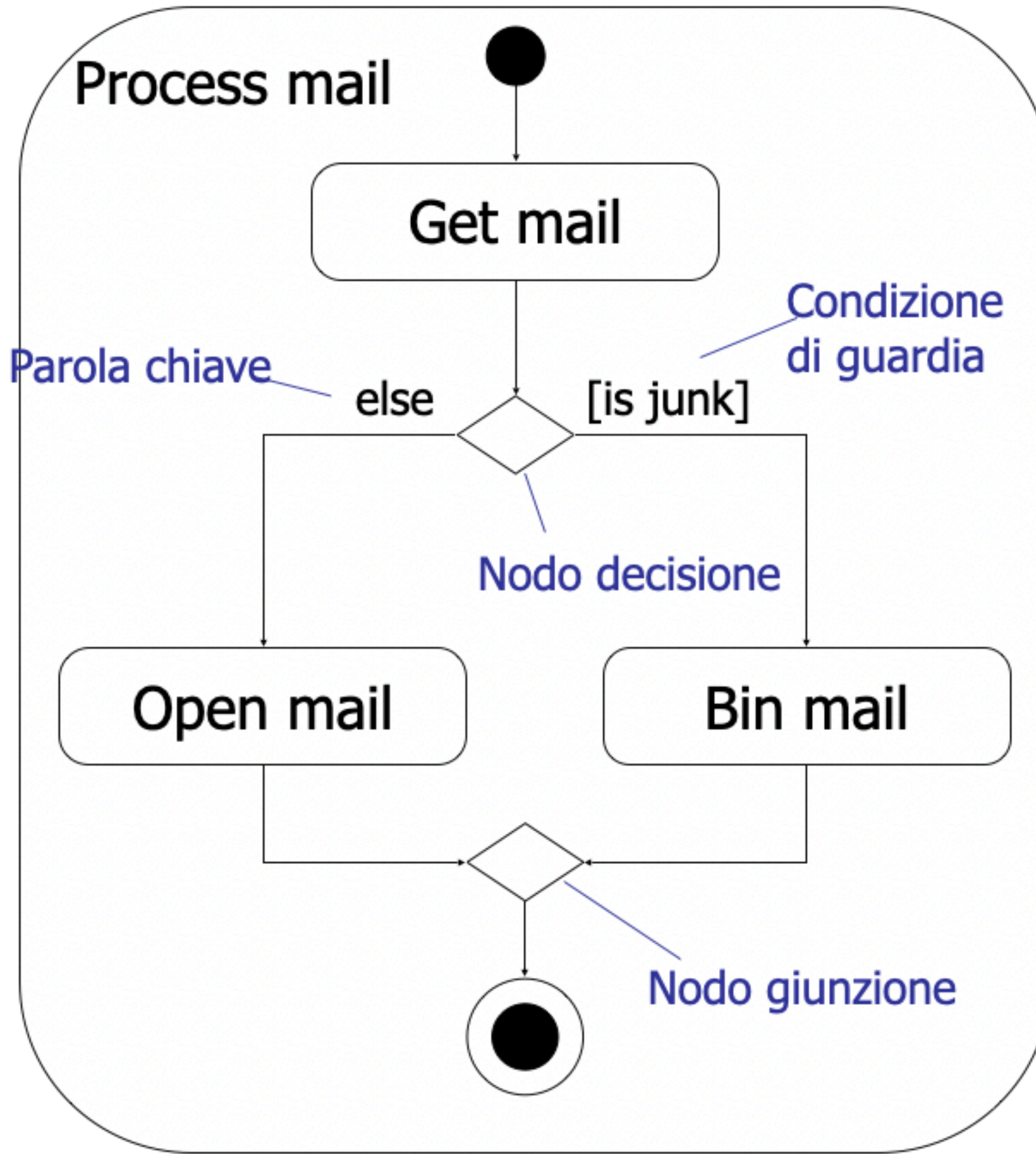
- Il **fork** rappresenta una biforcazione del flusso in due o più thread
 - ⦿ Rappresentato da un segmento orizzontale in cui entra un arco e ne esce più di uno
 - ⦿ Gli archi uscenti rappresentano thread paralleli avviati contemporaneamente ed eseguiti in modo concorrente (non sono etichettati)
- Il **join** è il duale del fork, sincronizza più flussi concorrenti
 - ⦿ Rappresentato da un segmento orizzontale in cui entra più di un arco e ne esce uno solo



FORK E JOIN

- ▶ Quando abbiamo un fork vuol dire che l'ordine di esecuzione tra i due flussi non é rilevante e possono anche avvenire in parallelo
- ▶ Quando abbiamo un join vuol dire che l'attività successiva può essere eseguita solo quando tutti i flussi in entrata hanno terminato la propria esecuzione (altrimenti viene eseguita appena arriva il primo dei flussi in entrata)







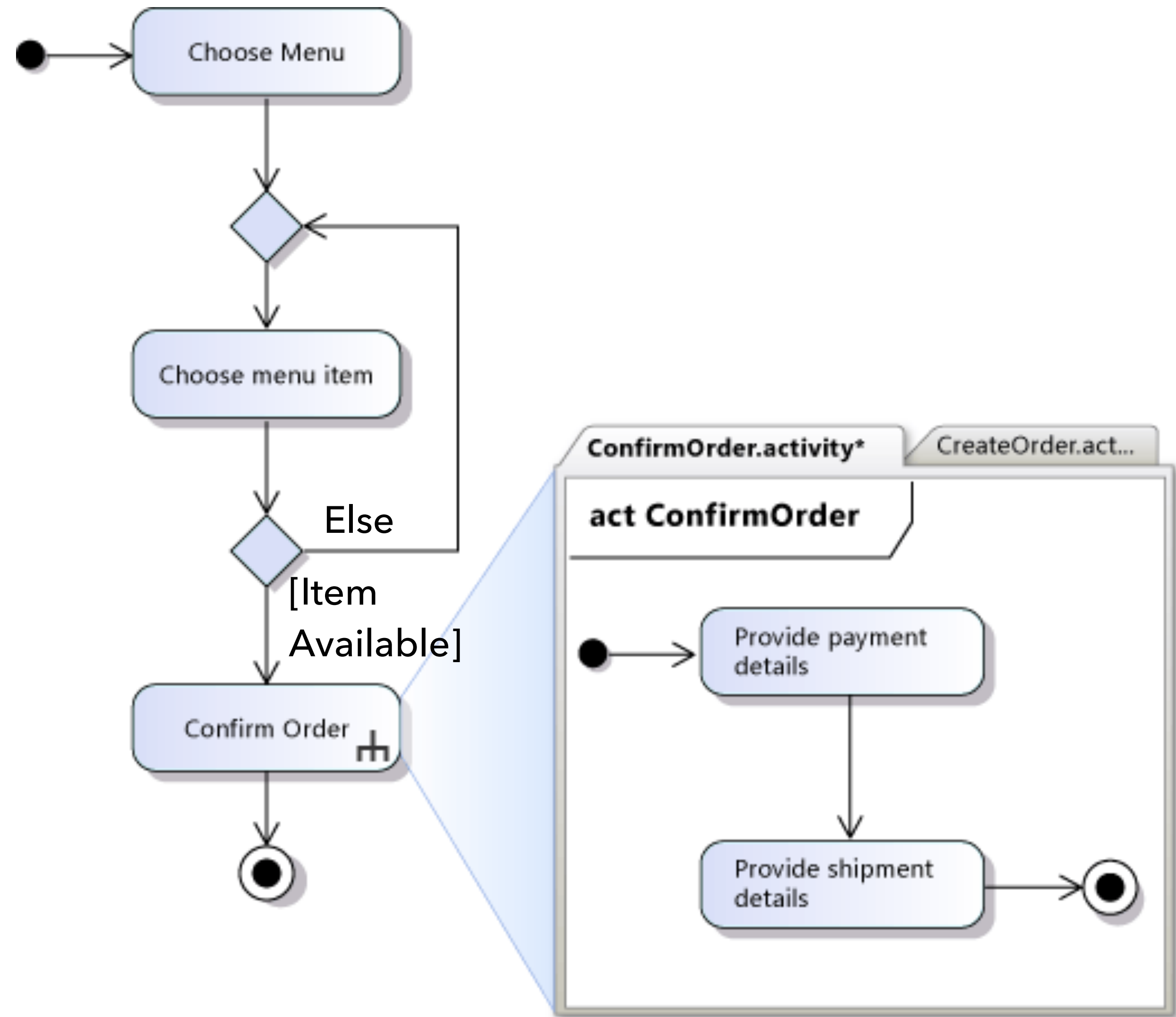
SCOMPOSIZIONE DI UN'AZIONE



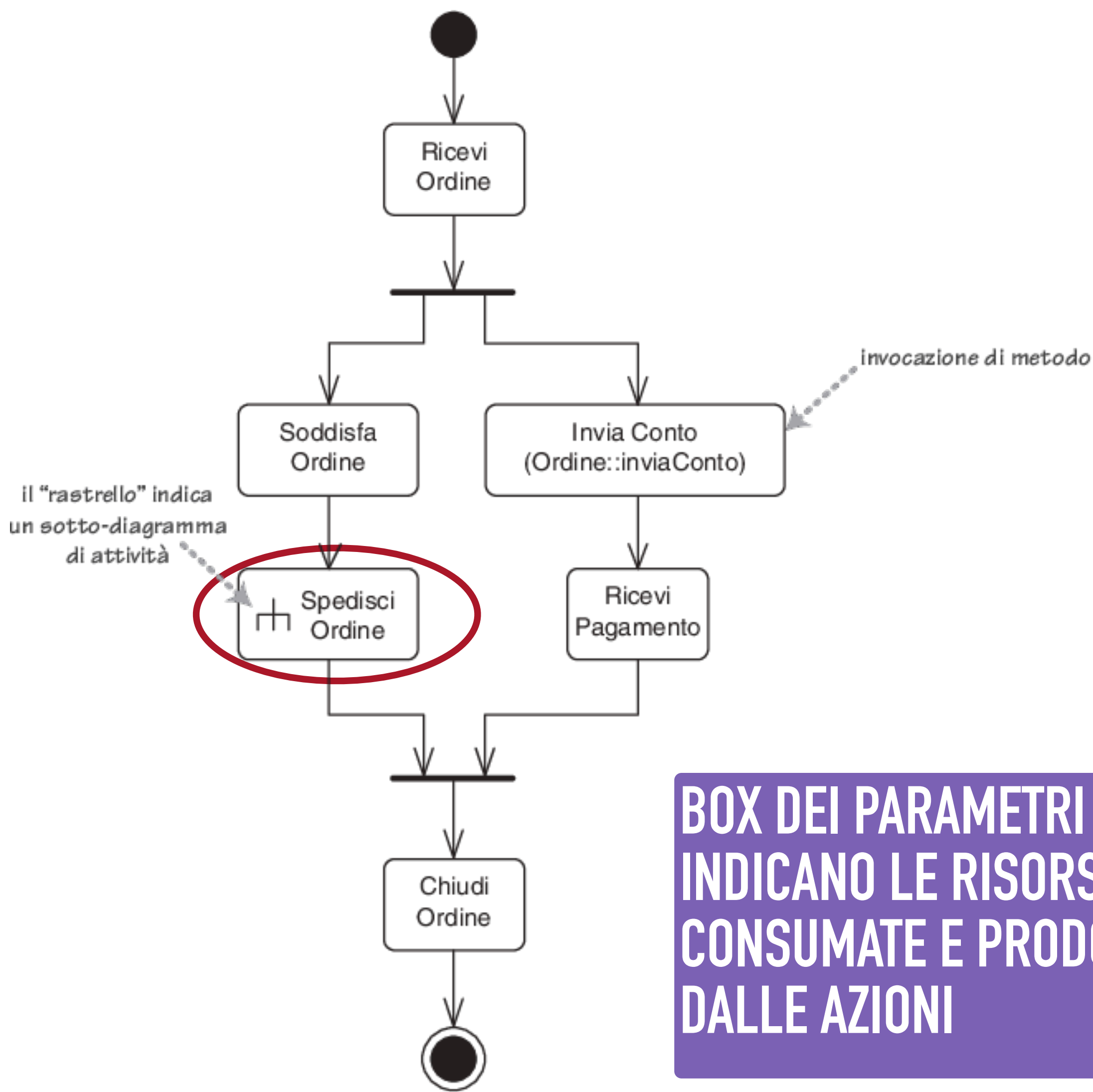
- La notazione precedentemente mostrata ha due limitazioni principali:
 - Non é rappresentato quale parte del software é responsabile di determinate azioni
 - In presenza di attività troppo complesse il diagramma diventa difficile da leggere

MACROATTIVITÀ E SOTTOATTIVITÀ

- ▶ Per supportare diversi livelli di astrazione, é possibile racchiudere porzioni di un'attività complesse in attività generiche (macroattività)
- ▶ Le azioni delle attività generiche saranno dettagliate in grafici separati
- ▶ Una macroattività é indicata con il simbolo del rastrello all'interno del rettangolo



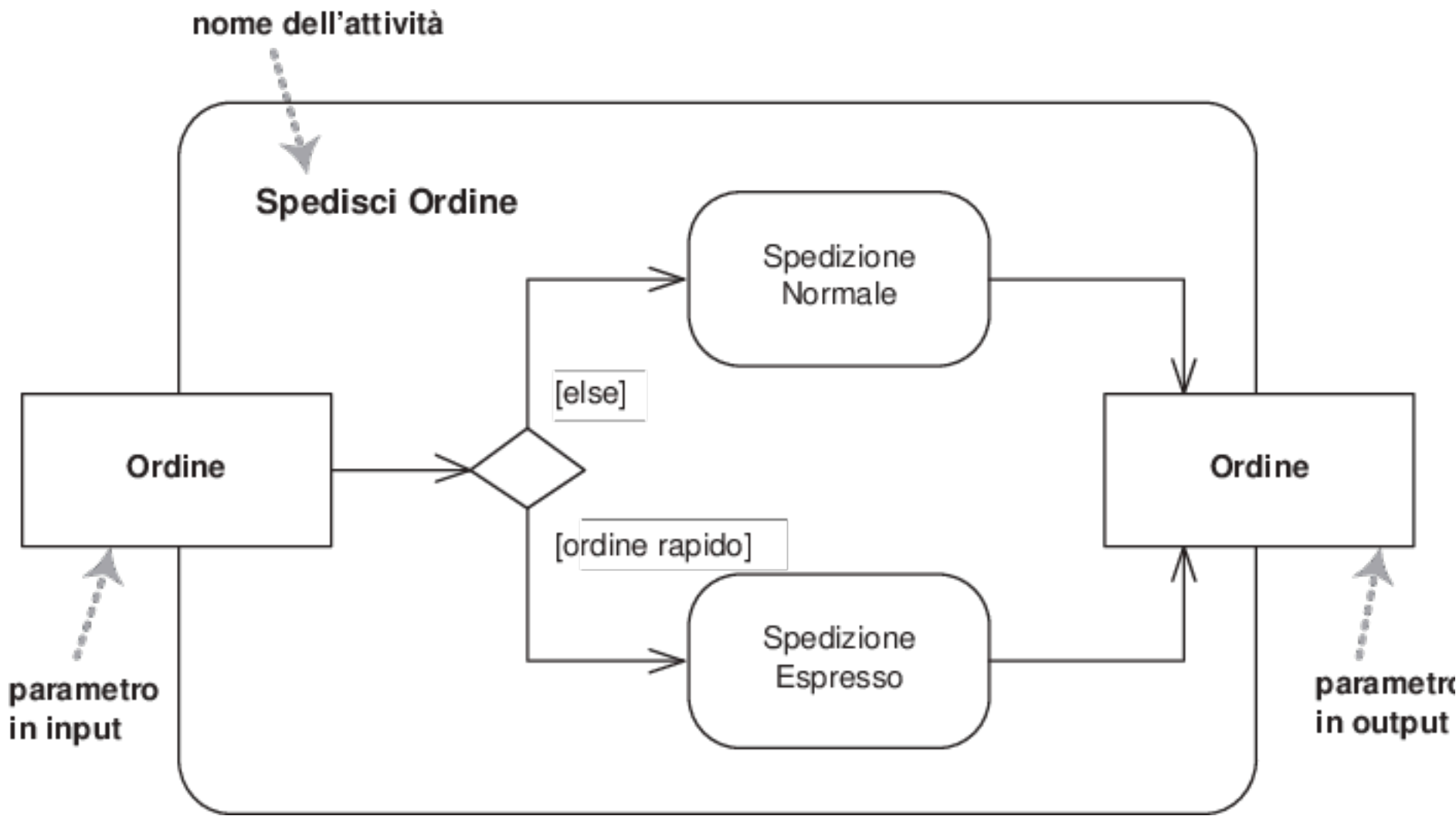
MACROATTIVITÀ E SOTTOATTIVITÀ



LE AZIONI POSSONO ESSERE ANCHE IMPLEMENTATE COME METODI DELLE CLASSI USANDO LA SINTASSI **NOMECLASSE::NOMEMETODO**

IN CASI ESTREMI, É POSSIBILE SCRIVERE UN'INTERO FRAMMENTO DI CODICE ALL'INTERNO DI UN SIMBOLO DI AZIONE

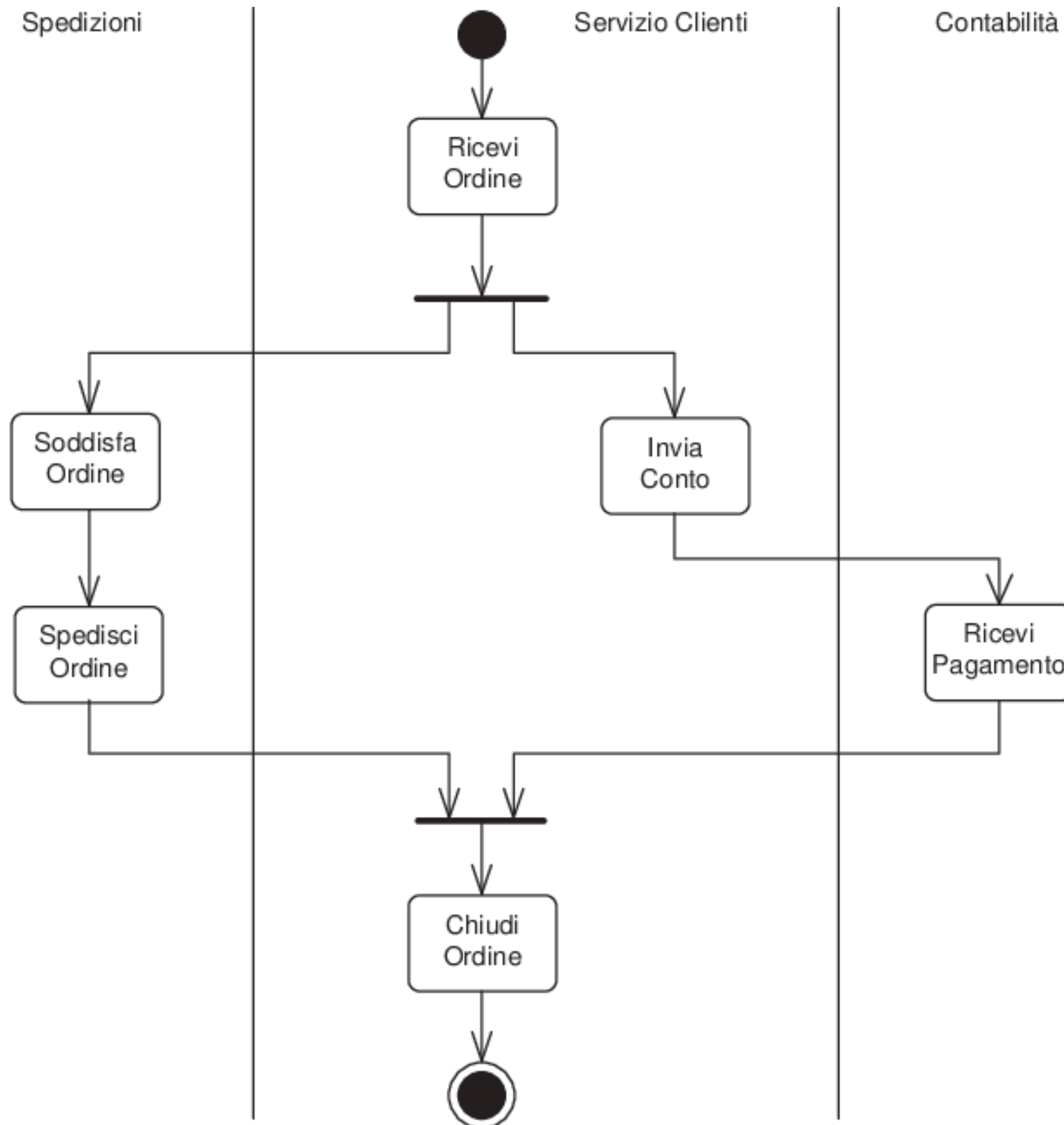
BOX DEI PARAMETRI INDICANO LE RISORSE CONSUMATE E PRODOTTE DALLE AZIONI





- Le partizioni esprimono "chi fa cosa" ossia quale classe/oggetto é responsabile per ciascuna azione o quale organizzazione esegue ciascuna azione del processo di business
- Il diagramma può essere diviso in partizioni da linee verticali dette **swimlanes** assegnate a ciascuna organizzazione o classe/oggetto
- Se un'azione si trova in una swimlane vuol dire che la sua responsabilità é assegnata alla classe/oggetto/organizzazione corrispondente
- Le swimlane permettono di modellare l'interazione tra diverse classi

PARTIZIONI





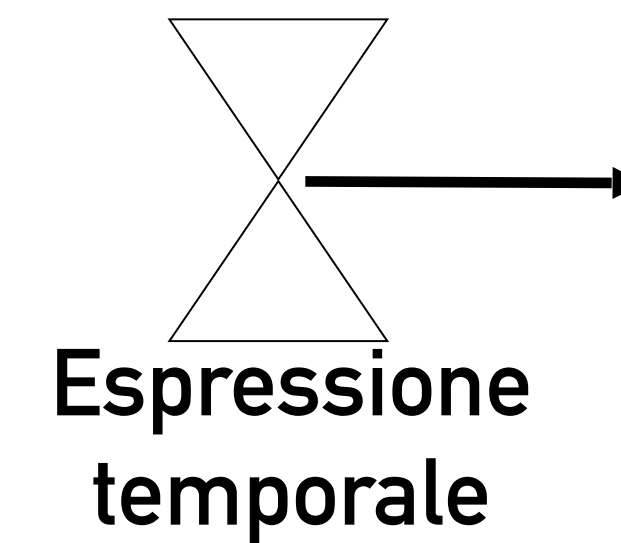
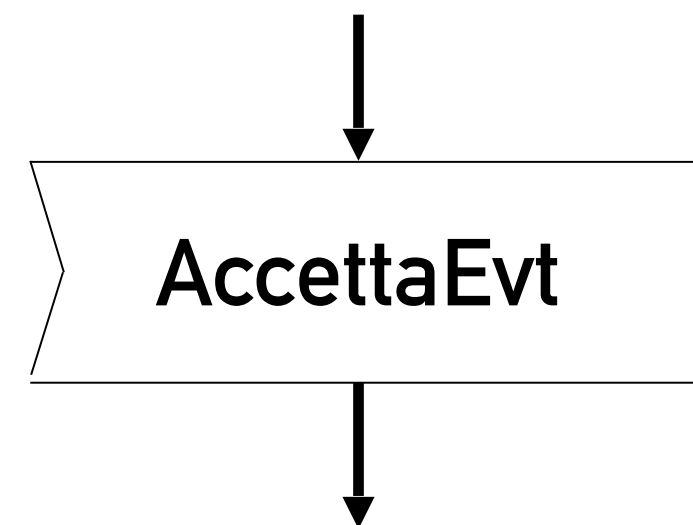
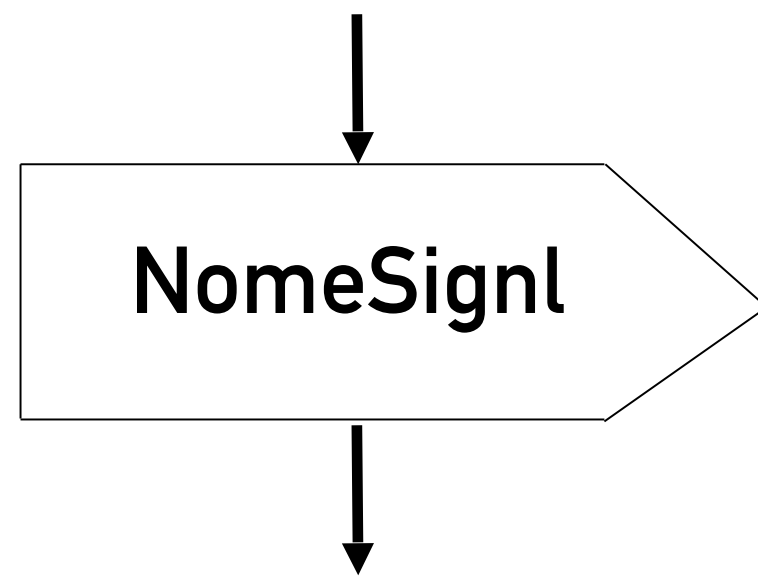
SEGNALI



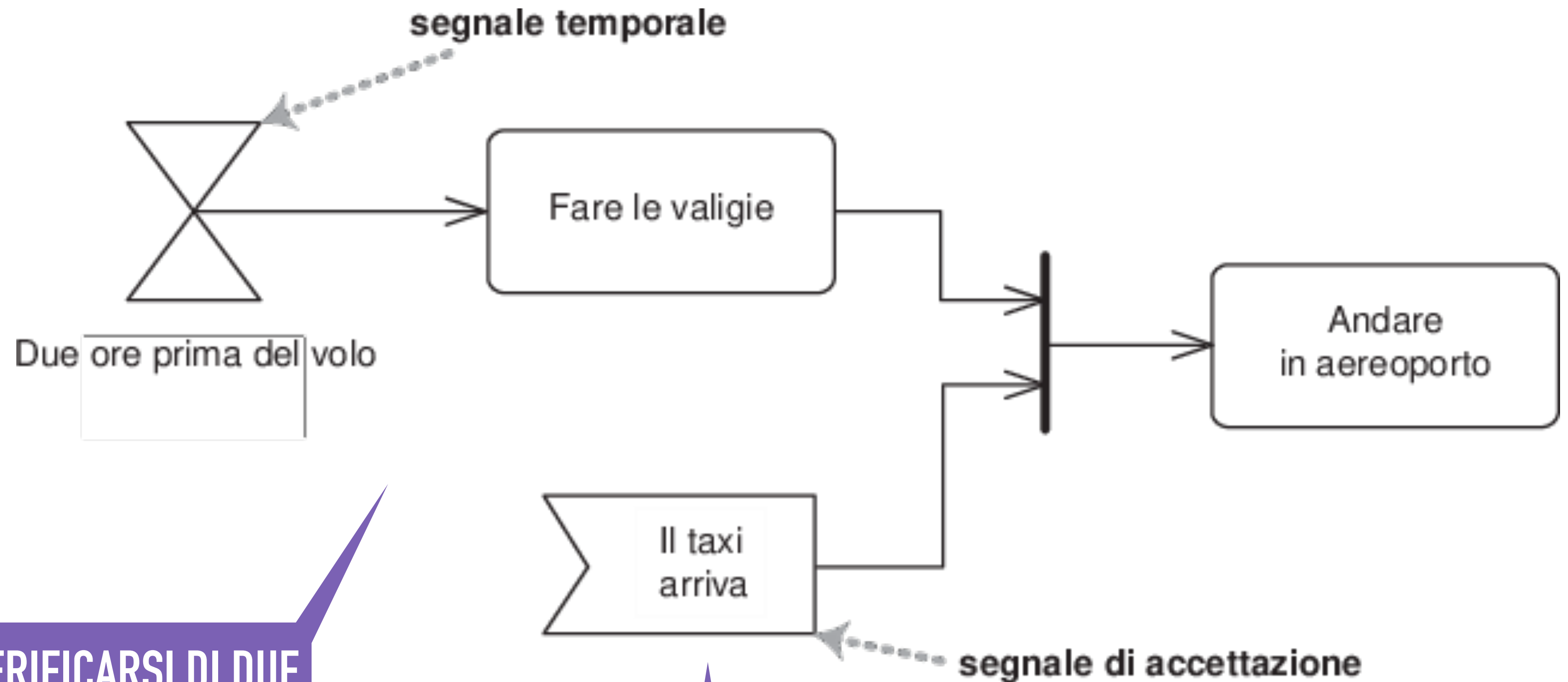
- A differenza degli statechart diagram, negli activity diagram non compaiono esplicitamente eventi
- Nei diagrammi visti finora il punto d'inizio corrisponde con una chiamata a metodo o con l'avvio di un programma
- I **segnali** rappresentano eventi diversi a cui possono rispondere le azioni
- Un segnale indica un qualsiasi evento asincrono proveniente da processi esterni che influenza l'esecuzione di un'activity



- Ci possono essere diversi tipi di segnali, indicati con simboli diversi
 - **Invio segnale:** azione di invio di un segnale asincrono
 - **Accetta evento:** aspetta che si verifichi l'evento specificato ed é attivato (si *mette in ascolto* e blocca il flusso di quel ramo) quando riceve l'evento. Appena riceve l'evento il flusso prosegue.
 - **Accetta evento temporale:** genera eventi secondo la sua espressione temporale



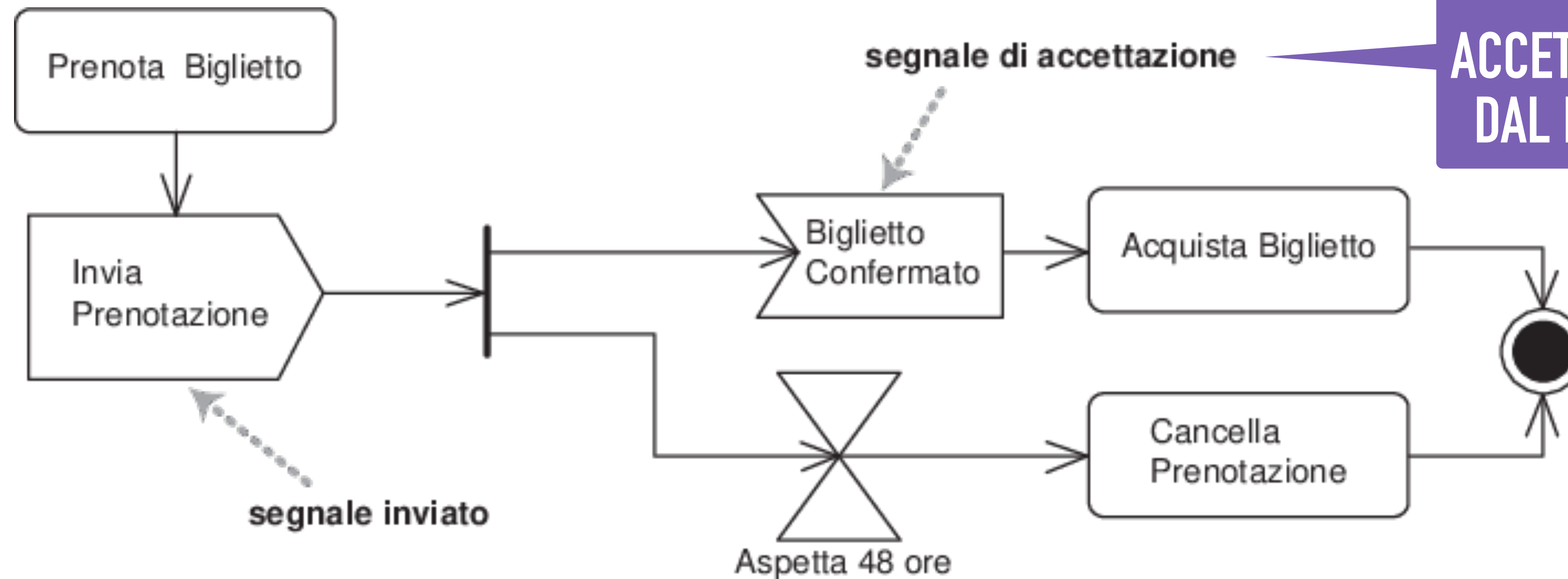
ESEMPIO: SEGNALI DI ACCETTAZIONE



ATTIVITÀ ATTENDE IL VERIFICARSI DI DUE SEGNALI ESTERNI, PERTANTO L'ATTIVITÀ È IN UNA SITUAZIONE "DI ASCOLTO"

SE NON CI SONO ARCHI IN ENTRATA È ATTIVATO (IN ASCOLTO DI EVENTI ESTERNI) QUANDO L'ATTIVITÀ CORRENTE INIZIA

ESEMPIO: RISPOSTA AI SEGNALI INVIATI



IN ASCOLTO DA QUANDO IL SEGNALE DI ACCETTAZIONE É ATTIVATO DAL FLUSSO ENTRANTE

L'INVIO DI UN SEGNALE PUÒ RAPPRESENTARE I CASI IN CUI SI DEVE ATTENDERE LA RISPOSTA A UN MESSAGGIO PRIMA DI CONTINUARE

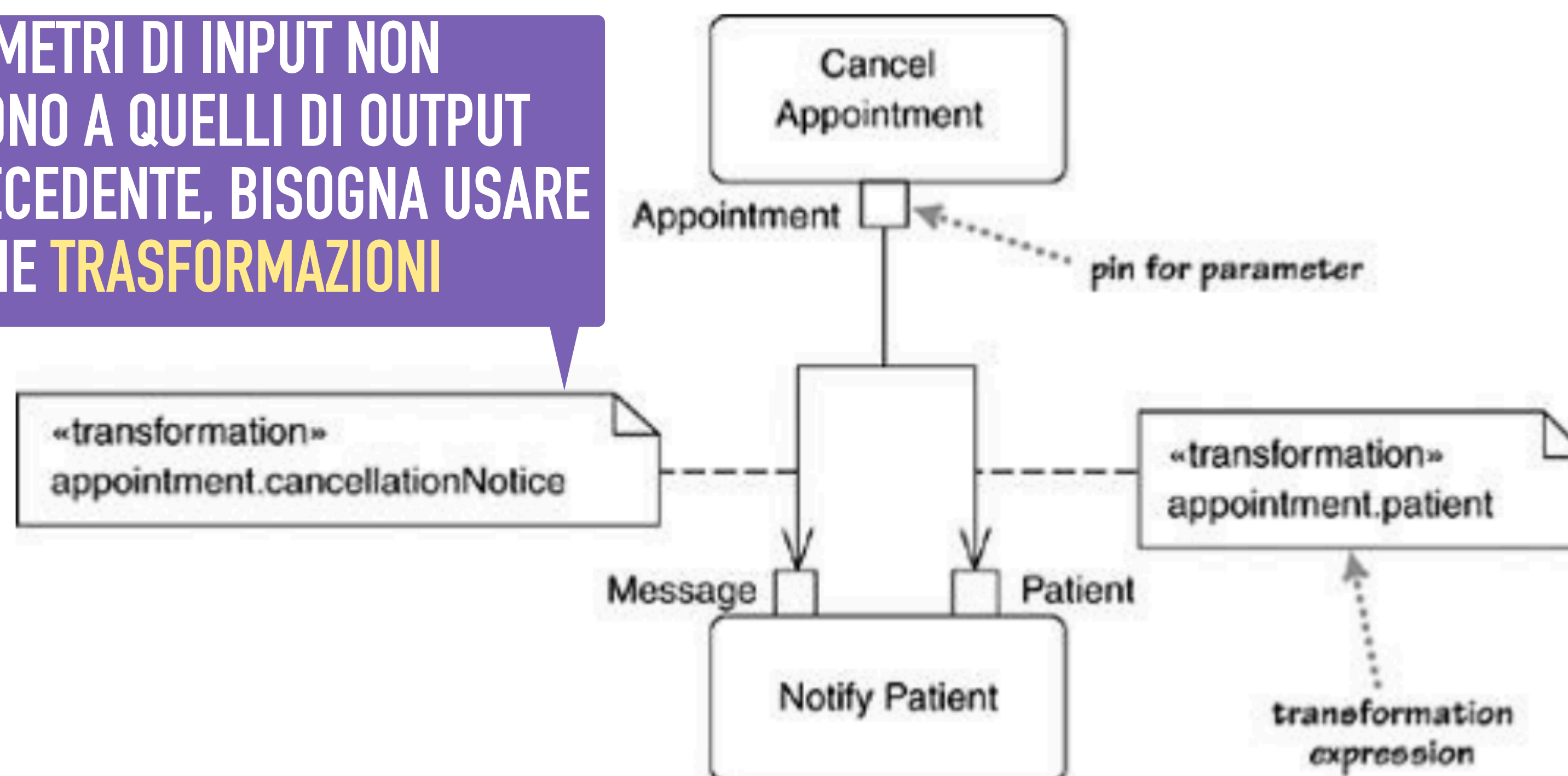
IL PRIMO DEI DUE FLUSSI ALTERNATIVI A RAGGIUNGERE LO STATO FINALE TERMINERÀ L'ATTIVITÀ (INFATTI NON ABBIAMO JOIN E QUINDI NON DOBBIAMO ATTENDERE LA TERMINAZIONE DI ENTRAMBI I FLUSSI)



PIN E TRASFORMAZIONI

- Utilizzati per esprimere i parametri delle azioni (dati richiesti e forniti dalle azioni)
- Corrispondono ai box dei parametri delle sottoattività

SE I PARAMETRI DI INPUT NON CORRISPONDONO A QUELLI DI OUTPUT DELL'AZIONE PRECEDENTE, BISOGNA USARE OPPORTUNE **TRASFORMAZIONI**





INVOCAZIONI MULTIPLE E REGIONI DI ESPANSIONE

INVOCAZIONI MULTIPLE



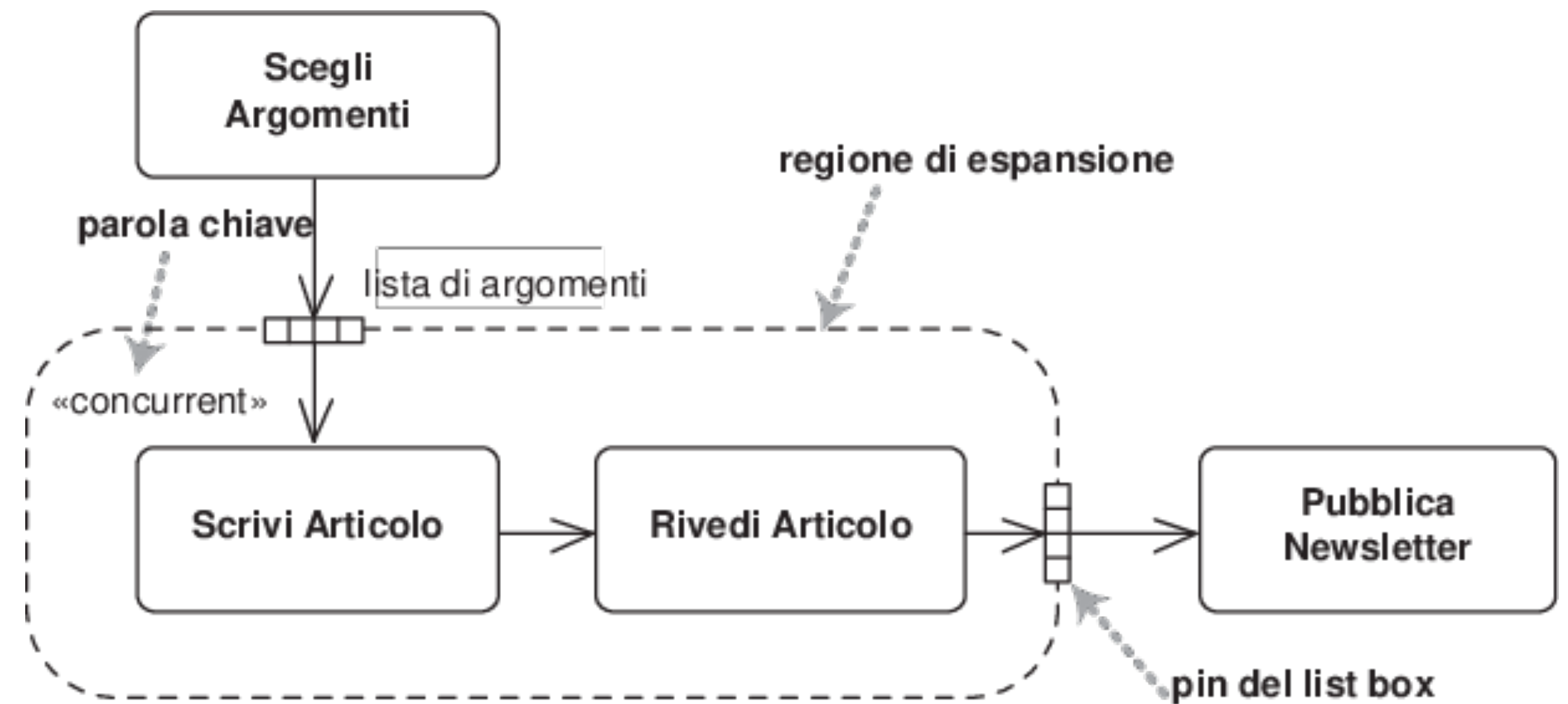
- L'output di un'azione può provocare l'esecuzione multipla di una o più azioni successive, una volta per ciascun elemento di una collezione
- La lista di argomenti in input e la lista di output sono riportate agli estremi dell'azione (gli output possono anche essere meno degli input, in caso di filtro)
- Se l'azione da invocare più volte é una sola si utilizza la seguente notazione compatta



REGIONI DI ESPANSIONE



- ▶ Una **regione di espansione** contraddistingue un'area dell'activity diagram in cui le azioni contenute vengono ripetute una volta per ciascun elemento di una collezione
- ▶ Se le azioni in una regione di espansione sono svolte in parallelo si usa la parola chiave `<<concurrent>>`, altrimenti si suppone che l'esecuzione sia sequenziale
- ▶ Nota: Nella notazione compatta l'esecuzione é parallela di default



-
- ```
graph TD; A[Scegli Argomenti] --> Fork(()); Fork -- lista di argomenti --> Fork; Fork -- «concurrent» --> B[Scrivi Articolo]; Fork -- «concurrent» --> C[Rivedi Articolo]; B --> C; C --> D{ }; D -- [accetta] --> Join(()); D -- [rifiuta] --> E((X)); E -.-> F[fine di flusso]; Join --> G[Pubblica Newsletter];
```



# CONCLUSIONI



## ▸ **Vantaggi:**

- Supportano la descrizione di comportamenti paralleli attraverso fork e join
- Compatibili con i diagrammi di flusso
- Permettono di descrivere processi e workflow

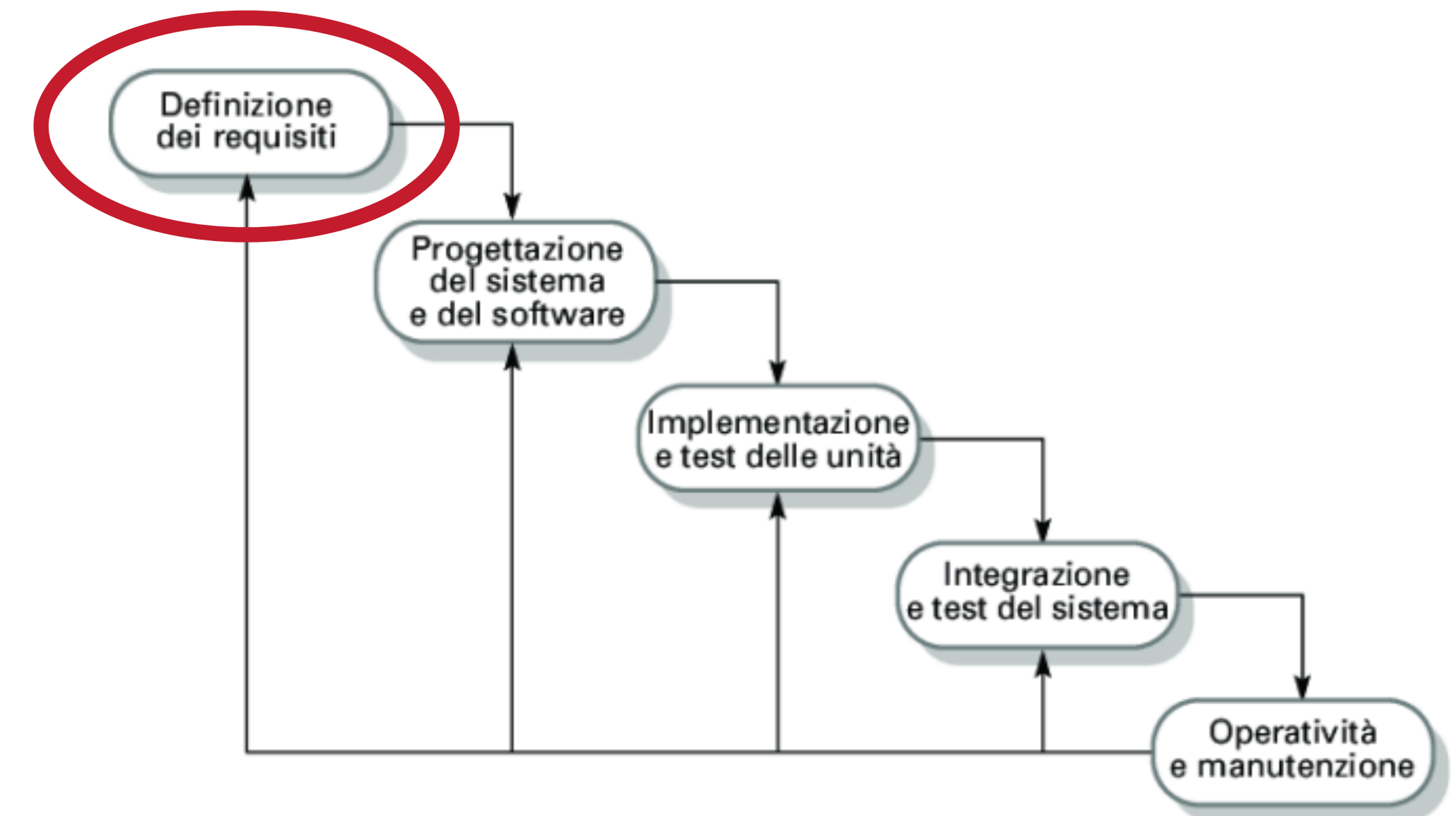
## ▸ **Svantaggi:**

- Meno diffusi dello pseudocodice per descrivere la logica comportamentale
- Possono essere utilizzati per descrivere gli scenari dei casi d'uso ma possono risultare poco comprensibili ai non esperti. In tali casi, si preferisce la descrizione testuale

# ACTIVITY DIAGRAM NELLA FASE DI ANALISI DEI REQUISITI



- In fase di analisi, un activity diagram può modellare il flusso di uno use case
- I casi d'uso possono essere descritti come un insieme di scenari alternativi in linguaggio naturale, ma tale descrizione é limitata per azioni che avvengono in parallelo
- I diagrammi delle attività rappresentano graficamente più scenari di uno use case (ossia più scenari paralleli che si sincronizzano)



# ACTIVITY DIAGRAM NELLA FASE DI ANALISI DEI REQUISITI



- In fase di progettazione di alto livello, modellano l'interazione tra le entità concettuali nella risoluzione di un problema
- In fase di progettazione di dettaglio, modellano un algoritmo che deve essere implementato
- Gli activity diagram possono anche modellare algoritmi concorrenti

